

BÁO CÁO ĐỒ ÁN CUỐI KÌ
MÔN HỌC: PHÂN TÍCH XỬ LÝ ẢNH
CHƯƠNG TRÌNH HỖ TRỢ CHẤM THI VÀ SỐ HÓA BÀI THI
TRẮC NGHIỆM

Nhóm thực hiện: Nhóm 10
Nguyễn Thế Minh Nhật

Mục lục

1	TỔNG QUAN ĐỀ TÀI	3
1.1	Đặt vấn đề	3
1.2	Mục tiêu đề tài	3
1.3	Phạm vi và Đối tượng nghiên cứu	3
1.3.1	Phạm vi nghiên cứu	3
1.3.2	Đối tượng nghiên cứu	4
1.4	Phương pháp tiếp cận	4
1.5	Cấu trúc báo cáo	4
2	CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	5
2.1	Tổng quan về Xử lý ảnh số (Digital Image Processing)	5
2.1.1	Ảnh xám (Grayscale)	5
2.1.2	Làm trơn ảnh (Gaussian Blur)	5
2.1.3	Phát hiện biên (Canny Edge Detection)	5
2.1.4	Biến đổi phối cảnh (Perspective Transform)	6
2.1.5	Phân ngưỡng thích nghi (Adaptive Thresholding)	6
2.2	Cơ sở lý thuyết về Nhận dạng quang học (Deep Learning OCR)	6
2.2.1	Mạng phát hiện văn bản (Text Detection): CRAFT	6
2.2.2	Mạng nhận dạng chuỗi (Text Recognition): CRNN + CTC	7
2.2.3	Hậu xử lý dữ liệu (Regex Post-processing)	9
2.3	Công nghệ và Thư viện nền tảng	9
2.3.1	OpenCV (Xử lý ma trận ảnh)	9
2.3.2	NumPy (Đại số tuyến tính)	9
2.3.3	ReportLab (Sinh dữ liệu lập trình)	9
2.4	Kiến trúc Phần mềm (Software Architecture)	9
3	PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	10
3.1	Kiến trúc mã nguồn (System Architecture)	10
3.1.1	Lớp Xử lý Logic (Core Logic Layer)	11
3.1.2	Lớp Trực quan hóa (Visualization Layer)	12
3.1.3	Lớp Điều phối và Cấu hình (Coordination & Configuration)	12
3.2	Thiết kế Quy trình hoạt động (Workflow Design)	12
3.3	Thiết kế Dữ liệu (Data Structures)	13
3.3.1	Cấu trúc Template (coordinates.json)	13
3.3.2	Cấu trúc Đáp án (answer_key.csv)	14
4	CÀI ĐẶT VÀ HIỆN THỰC	15
4.1	Module Sinh phiếu thi tự động (Auto Sheet Generator)	15
4.2	Module Xử lý ảnh tiền kỳ (Preprocessing)	15
4.2.1	Thuật toán phát hiện khung giấy	15
4.2.2	Biến đổi phối cảnh (Perspective Transform)	15
4.3	Module Chấm thi (OMR Engine)	16
4.3.1	Kỹ thuật Phân ngưỡng thích nghi (Adaptive Thresholding)	16
4.3.2	Logic xác định ô tô (Bubble Detection)	16
4.4	Module Nhận dạng chữ (OCR Engine)	17
4.4.1	Quy trình xử lý	17
4.4.2	Hậu xử lý dữ liệu bằng Regex	17

4.5	Tinh chỉnh Mô hình Nhận dạng (OCR Fine-tuning)	17
4.5.1	Chuẩn bị dữ liệu	17
4.5.2	Cấu hình Huấn luyện	18
4.5.3	Xử lý bất đồng bộ kiến trúc (Selective Weight Loading)	18
4.6	Cơ chế Nhận dạng Kép và Chiến lược Hợp nhất (Hybrid OCR Strategy)	18
4.6.1	Phân tích hiệu năng so sánh	18
4.6.2	Cơ chế quyết định (Decision Logic)	18
5	KẾT QUẢ THỰC NGHIỆM	20
5.1	Môi trường và Dữ liệu kiểm thử	20
5.1.1	Thiết lập kịch bản (Test Setup)	20
5.1.2	Tham số cấu hình	20
5.2	Kết quả Huấn luyện Mô hình (Training Results)	20
5.3	Đánh giá Hiệu năng Nhận dạng (Evaluation)	21
5.3.1	So sánh hiệu năng OCR (Ablation Study)	21
5.3.2	Kết quả chấm thi (OMR)	21
5.4	Kết quả Xử lý ảnh (Preprocessing Results)	21
5.4.1	Biến đổi hình học (Warp Perspective)	21
5.4.2	Xử lý nhị phân và Khử nhiễu	22
5.5	Kết quả Chấm thi (OMR Results)	22
5.5.1	Nhật ký xử lý (Console Log)	22
5.5.2	Trực quan hóa kết quả (Visualization)	23
5.6	Kết quả Nhận dạng thông tin (OCR Results)	24
5.7	Tổng kết Hiệu năng và Hạn chế	25
5.7.1	Hiệu năng đạt được	25
5.7.2	Hạn chế tồn tại	25
6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	26
6.1	Kết luận	26
6.1.1	Kết quả đạt được	26
6.1.2	Bài học kinh nghiệm	26
6.2	Hướng phát triển trong tương lai	26
	Tài liệu tham khảo	27

Chương 1

TỔNG QUAN ĐỀ TÀI

1.1 Đặt vấn đề

Trong lĩnh vực Thị giác máy tính (Computer Vision), bài toán xử lý phiếu thi trắc nghiệm (OMR) là một chủ đề kinh điển, thường được sử dụng để minh họa cho các kỹ thuật xử lý ảnh cơ bản như nhị phân hóa, phát hiện biên và biến đổi hình học.

Tuy nhiên, khi áp dụng vào thực tế với thiết bị phổ thông như camera điện thoại, bài toán này nảy sinh nhiều thách thức kỹ thuật thú vị mà các phương pháp truyền thống khó giải quyết trọn vẹn, ví dụ như:

1. **Chất lượng ảnh đầu vào không đồng đều:** Ảnh chụp thường bị nghiêng, méo, hoặc chịu ảnh hưởng của điều kiện ánh sáng (bóng đổ, lóa sáng) khiến việc phân ngưỡng (thresholding) trở nên khó khăn.
2. **Vấn đề nhận dạng chữ viết tay:** Ngoài việc chấm điểm trắc nghiệm, nhu cầu số hóa các thông tin viết tay (như Số báo danh) đòi hỏi sự kết hợp với các mô hình Học sâu (Deep Learning) thay vì chỉ xử lý ảnh đơn thuần.

Với mong muốn vận dụng các kiến thức đã học trong môn Phân tích Xử lý ảnh vào một bài toán thực tế cụ thể, nhóm quyết định thực hiện đề tài **"SmartGrader"**. Đây là một nỗ lực bước đầu nhằm xây dựng chương trình hỗ trợ chấm thi quy mô nhỏ, đồng thời là cơ hội để nhóm rèn luyện kỹ năng lập trình và tư duy giải quyết vấn đề.

1.2 Mục tiêu đề tài

Mục tiêu cốt lõi của đồ án không dừng lại ở việc tạo ra một phần mềm hoàn chỉnh, mà tập trung vào việc **thử nghiệm và đánh giá hiệu quả** của các giải thuật xử lý ảnh khi áp dụng vào dữ liệu thật. Các mục tiêu cụ thể bao gồm:

- **Tự động hóa việc định nghĩa Template (Auto Generator):** Thay thế phương pháp đo đặc tọa độ thủ công (để sai sót) bằng cách lập trình sinh phiếu thi tự động. Mục tiêu là tạo ra bộ dữ liệu mẫu (Ground Truth) có độ tin cậy cao để phục vụ quá trình chấm.
- **Cải thiện thuật toán OMR:** Tìm hiểu và so sánh hiệu quả của kỹ thuật Phân ngưỡng thích nghi (Adaptive Thresholding) so với phân ngưỡng toàn cục trong việc xử lý các ảnh bị nhiễu hoặc ánh sáng không đều [?].
- **Thử nghiệm công nghệ OCR:** Bước đầu tìm hiểu cách tích hợp thư viện EasyOCR để nhận dạng các trường thông tin viết tay (Số báo danh), kết hợp với các bộ lọc dữ liệu (Regex) để đánh giá khả năng "làm sạch" kết quả của mô hình [?].
- **Xây dựng quy trình xử lý (Pipeline):** Tổ chức mã nguồn thành một luồng xử lý khép kín: từ ảnh đầu vào → tiền xử lý → nhận dạng → xuất kết quả, giúp rèn luyện tư duy thiết kế hệ thống.

1.3 Phạm vi và Đối tượng nghiên cứu

1.3.1 Phạm vi nghiên cứu

Do giới hạn về thời gian và nguồn lực, đề tài tập trung giải quyết các bài toán cụ thể sau:

- **Dữ liệu đầu vào:** Chương trình chỉ xử lý các tệp tin hình ảnh (.jpg, .png) hoặc tài liệu quét (.pdf) của phiếu trả lời trắc nghiệm do chính chương trình sinh ra (theo Template định sẵn).
- **Công nghệ sử dụng:** Mã nguồn được viết bằng ngôn ngữ **Python**, kết hợp các thư viện mã nguồn mở phổ biến: **OpenCV** (cho xử lý ảnh), **EasyOCR** (cho nhận dạng ký tự), và **ReportLab** (để tạo phiếu thi mẫu).
- **Giới hạn kỹ thuật:** Chương trình yêu cầu ảnh đầu vào phải nhìn thấy rõ 4 góc định vị (neo) của phiếu thi để thực hiện phép biến đổi hình học (Warp Perspective). Đề tài chưa xử lý các trường hợp phiếu thi bị rách, mất góc hoặc biến dạng quá lớn [5].

1.3.2 Đối tượng nghiên cứu

Đồ án tập trung tìm hiểu và vận dụng các đối tượng kiến thức sau:

- **Các thuật toán xử lý ảnh cơ bản:** Phát hiện biên (Canny Edge Detection), Tìm đường bao (Contour Retrieval) và Biến đổi phối cảnh (Perspective Transform).
- **Kỹ thuật nhị phân hóa:** So sánh hiệu quả giữa Otsu's Binarization và Adaptive Gaussian Thresholding trong điều kiện ánh sáng thực tế.
- **Công nghệ OCR:** Tìm hiểu cấu trúc và cách fine-tuning mô hình EasyOCR để nhận dạng chữ số viết tay.

1.4 Phương pháp tiếp cận

Để mã nguồn dễ dàng quản lý và bảo trì, nhóm áp dụng tư duy tổ chức mã nguồn theo hướng **Module hóa (Modular Design)**, chia tách các thành phần xử lý riêng biệt:

1. **Module Xử lý chính (Core Processing):** Bao gồm `omr_engine.py` (thực hiện thuật toán chấm điểm) và `ocr_engine.py` (gọi thư viện nhận dạng ký tự). Các tham số vị trí được tách riêng ra file cấu hình `coordinates.json` thay vì viết cứng trong code (Hard-coding).
2. **Module Hiển thị (Visualization):** File `renderer.py` chịu trách nhiệm vẽ lại kết quả chấm (vòng tròn xanh/đỏ) lên ảnh gốc để người dùng dễ quan sát.
3. **Module Điều phối (Main Controller):** File `main.py` đóng vai trò kết nối, đọc dữ liệu ảnh đầu vào, gọi lần lượt các hàm xử lý và xuất kết quả cuối cùng ra màn hình hoặc file Excel.

1.5 Cấu trúc báo cáo

Báo cáo được trình bày trong 6 chương với nội dung cụ thể như sau:

- **Chương 1: Tổng quan đề tài.** Trình bày lý do chọn đề tài, mục tiêu, phạm vi và phương pháp tiếp cận của nhóm.
- **Chương 2: Cơ sở lý thuyết.** Tóm tắt các kiến thức nền tảng về Xử lý ảnh (Grayscale, Edge Detection, Warping) và công nghệ OCR được sử dụng trong đồ án.
- **Chương 3: Phân tích và Thiết kế hệ thống.** Mô tả luồng dữ liệu (Flowchart) và cách tổ chức thư mục, cấu trúc dữ liệu đầu vào/đầu ra.
- **Chương 4: Cài đặt và Hiện thực.** Đi sâu vào chi tiết cài đặt các thuật toán quan trọng: Sinh đề tự động, Chống nhiễu bằng Adaptive Threshold, và Hậu xử lý dữ liệu OCR.
- **Chương 5: Kết quả thực nghiệm.** Trình bày các kịch bản kiểm thử trên ảnh chụp thực tế và đánh giá sơ bộ độ chính xác của chương trình.
- **Chương 6: Kết luận và Hướng phát triển.** Tổng kết các kiến thức đã học được, nhìn nhận các hạn chế còn tồn tại và đề xuất hướng cải tiến trong tương lai.

Chương 2

CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1 Tổng quan về Xử lý ảnh số (Digital Image Processing)

Trong khuôn khổ đồ án, các kỹ thuật xử lý ảnh đóng vai trò tiền xử lý (Preprocessing) nhằm chuẩn hóa dữ liệu đầu vào. Mục tiêu là biến đổi ảnh chụp thô (raw image) thành dạng ảnh chuẩn mà máy tính có thể "hiểu" được. Các kỹ thuật chính được áp dụng bao gồm:

2.1.1 Ảnh xám (Grayscale)

Ảnh xám là ảnh mà mỗi điểm ảnh (pixel) chỉ mang thông tin về cường độ sáng (Luminance), loại bỏ hoàn toàn thông tin màu sắc (Chrominance).

Việc chuyển đổi từ không gian màu RGB sang Grayscale là bước bắt buộc vì hai lý do:

- **Giảm độ phức tạp tính toán:** Giảm dữ liệu từ 3 kênh màu xuống 1 kênh, giúp tăng tốc độ xử lý lên gấp 3 lần.
- **Tăng hiệu quả phát hiện biên:** Các thuật toán tìm biên hoạt động dựa trên sự thay đổi cường độ sáng, việc giữ lại màu sắc thường gây nhiễu không cần thiết.

Trong mã nguồn (`src/utils/image_utils.py`), nhóm sử dụng công thức chuyển đổi tiêu chuẩn dựa trên độ nhạy của mắt người:

$$Y = 0.299 \times R + 0.587 \times G + 0.114 \times B \quad (2.1)$$

Hàm sử dụng: `cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)`.

2.1.2 Làm trơn ảnh (Gaussian Blur)

Ảnh chụp thực tế luôn tồn tại nhiễu tần số cao (high-frequency noise) như nhiễu hạt (salt-and-pepper). Nếu không xử lý, các nhiễu này sẽ bị thuật toán Canny nhận nhầm là "biên".

Đồ án sử dụng bộ lọc Gaussian (Gaussian Filter) đóng vai trò là một bộ lọc thông thấp (low-pass filter). Ảnh được tích chập (convolution) với một hạt nhân (kernel) 5×5 . Giá trị pixel mới là trung bình có trọng số của các pixel lân cận, trong đó các pixel gần tâm có trọng số cao hơn.

2.1.3 Phát hiện biên (Canny Edge Detection)

Thay vì dùng các toán tử đơn giản như Sobel (dễ bị đứt nét), nhóm sử dụng thuật toán Canny nhờ khả năng tối ưu hóa việc phát hiện biên thông qua quy trình 4 bước chặt chẽ:

1. **Khử nhiễu:** Sử dụng Gaussian Blur như đã trình bày ở trên.
2. **Tính Gradient:** Xác định cường độ và hướng thay đổi của độ sáng tại mỗi điểm.
3. **Non-maximum Suppression (Triệt tiêu phi cực đại):** Kỹ thuật này giúp làm mảnh nét biên. Thuật toán sẽ loại bỏ các điểm pixel không phải là cực đại cục bộ theo hướng gradient, đảm bảo đường biên chỉ rộng 1 pixel.

4. **Hysteresis Thresholding (Phân ngưỡng trễ):** Sử dụng hai ngưỡng (threshold1 và threshold2).

- Điểm có cường độ $> T_{cao}$: Chắc chắn là biên (Strong Edge).
- Điểm nằm giữa $[T_{thap}, T_{cao}]$: Chỉ được coi là biên nếu nó kết nối với một điểm "Strong Edge". Cơ chế này giúp thuật toán nối liền các đường biên bị đứt quãng do ánh sáng yếu.

2.1.4 Biến đổi phối cảnh (Perspective Transform)

Để xử lý hiện tượng méo hình thang do góc chụp nghiêng, hệ thống cần thực hiện phép biến đổi phối cảnh để đưa ảnh về góc nhìn trực diện (Top-down view).

Về mặt toán học, đây là quá trình tìm Ma trận Homography M kích thước 3×3 . Do ma trận này có 8 bậc tự do (Degrees of Freedom), hệ thống yêu cầu tối thiểu 4 cặp điểm tương ứng (4 góc phiếu thi) để giải hệ phương trình:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (2.2)$$

Module `image_utils.py` chịu trách nhiệm tính toán ma trận này và ánh xạ lại toàn bộ pixel sang hệ tọa độ mới.

2.1.5 Phân ngưỡng thích nghi (Adaptive Thresholding)

Các phương pháp phân ngưỡng toàn cục (Global Thresholding) như Otsu giả định ánh sáng phân bố đều trên toàn ảnh. Điều này không đúng với ảnh chụp thực tế có bóng đổ.

Nhóm lựa chọn Adaptive Gaussian Thresholding. Ngưỡng $T(x, y)$ tại mỗi điểm ảnh không cố định mà được tính dựa trên vùng lân cận:

$$T(x, y) = \left(\sum_{i=-k}^k \sum_{j=-k}^k w(i, j) \cdot I(x + i, y + j) \right) - C \quad (2.3)$$

Trong đó, $w(i, j)$ là trọng số Gaussian (các điểm gần tâm quan trọng hơn). Việc trừ đi hằng số C giúp tinh chỉnh độ nhạy, đảm bảo tách biệt được vết mực ngay cả khi nền giấy tại vùng đó bị tối hơn các vùng khác.

2.2 Cơ sở lý thuyết về Nhận dạng quang học (Deep Learning OCR)

Để giải quyết bài toán số hóa thông tin thí sinh (Tên, Số báo danh), đề án sử dụng thư viện EasyOCR. Khác với các công cụ OCR truyền thống (như Tesseract) dựa trên các đặc trưng thủ công, EasyOCR là một hệ thống End-to-End Deep Learning, kết hợp hai mạng nơ-ron chuyên biệt cho hai tác vụ: Phát hiện (Detection) và Nhận dạng (Recognition).

2.2.1 Mạng phát hiện văn bản (Text Detection): CRAFT

Để định vị vị trí các dòng chữ trong ảnh, EasyOCR sử dụng kiến trúc CRAFT (Character Region Awareness for Text Detection).

Khác với các mô hình phát hiện vật thể (Object Detection) như YOLO hay SSD thường dùng khung bao hình chữ nhật (Bounding Box), CRAFT tiếp cận bài toán theo hướng phân đoạn ngữ nghĩa (Segmentation). Mô hình này dự đoán hai loại bản đồ nhiệt (heatmap) cho từng pixel:

- **Region Score:** Xác suất pixel đó nằm trong vùng của một ký tự.
- **Affinity Score:** Xác suất pixel đó là cầu nối (liên kết) giữa hai ký tự liền kề.

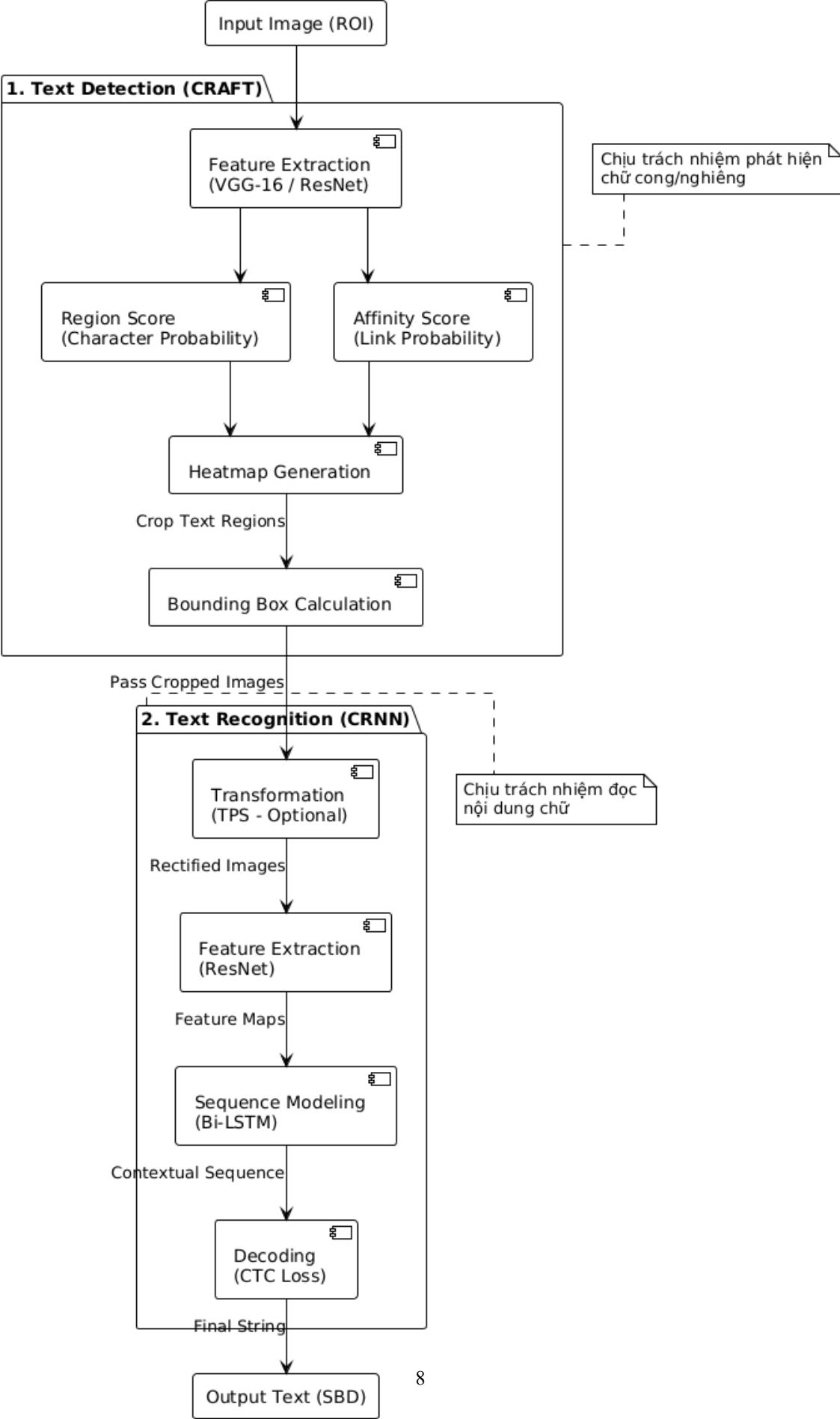
Ưu điểm: Nhờ cơ chế liên kết (Affinity), CRAFT có khả năng phát hiện cực tốt các dòng văn bản bị cong, nghiêng hoặc biến dạng hình học - điều rất phổ biến khi chụp phiếu thi bằng điện thoại.

2.2.2 Mạng nhận dạng chuỗi (Text Recognition): CRNN + CTC

Sau khi đã cắt được vùng ảnh chứa văn bản (ROI), bài toán trở thành chuyển đổi hình ảnh sang chuỗi ký tự (Image-to-Sequence). Kiến trúc CRNN (Convolutional Recurrent Neural Network) được áp dụng gồm 3 tầng xử lý nối tiếp:

1. **Trích xuất đặc trưng (Feature Extraction - CNN):** Sử dụng mạng ResNet (Residual Network) để trích xuất các bản đồ đặc trưng (feature maps) từ ảnh đầu vào. Tầng này giúp mô hình "nhìn" thấy các nét gạch, đường cong của chữ.
2. **Mô hình hóa chuỗi (Sequence Modeling - RNN):** Các đặc trưng được đưa vào mạng Bi-LSTM (Bidirectional Long Short-Term Memory). Mạng này quét chuỗi đặc trưng theo cả hai chiều (trái sang phải và phải sang trái) để học ngữ cảnh.
Ví dụ: Việc biết ký tự trước đó là 'P' và sau đó là 'X' sẽ giúp mô hình dự đoán ký tự giữa là 'T' (trong từ PTXLA) chính xác hơn.
3. **Giải mã (Decoding - CTC Loss):** Tầng cuối cùng sử dụng thuật toán CTC (Connectionist Temporal Classification) để giải mã đầu ra của RNN thành chuỗi ký tự cuối cùng. CTC giải quyết vấn đề bất đồng bộ giữa kích thước ảnh đầu vào (width) và độ dài chuỗi ký tự đầu ra (text length) mà không cần gióng hàng (alignment) thủ công.

EasyOCR Internal Architecture (Deep Dive)



Hình 2.1: Kiến trúc CRNN: Sự kết hợp giữa CNN (thị giác) và RNN (ngôn ngữ)

2.2.3 Hậu xử lý dữ liệu (Regex Post-processing)

Mô hình Deep Learning hoạt động dựa trên xác suất, do đó đầu ra thường chứa nhiễu (Stochastic Noise). Để đảm bảo tính toàn vẹn dữ liệu (Data Integrity), hệ thống sử dụng Biểu thức chính quy (Regular Expression) như một bộ lọc tất định (Deterministic Filter).

Chiến lược áp dụng cho từng trường thông tin:

- **Trường Số báo danh (Numeric Field):** Sử dụng pattern `\D` (Non-digit) để tìm và loại bỏ tất cả các ký tự không phải là số. Điều này giúp khắc phục lỗi phổ biến khi OCR nhận diện nhầm vết bẩn thành dấu chấm câu.
- **Trường Họ tên (Text Field):** Sử dụng pattern `[^\w\s]` để loại bỏ các ký tự đặc biệt, chỉ giữ lại chữ cái và khoảng trắng, đảm bảo tên thí sinh đúng định dạng văn bản.

2.3 Công nghệ và Thư viện nền tảng

2.3.1 OpenCV (Xử lý ma trận ảnh)

OpenCV không chỉ là công cụ đọc ảnh, mà là thư viện hiện thực hóa các thuật toán Toán học thị giác máy tính. Trong đồ án, OpenCV chịu trách nhiệm thực hiện các phép biến đổi không gian (Spatial Transformations) và phân tích cấu trúc tô pô (Topological Structure Analysis) thông qua việc tìm đường bao (Contours).

2.3.2 NumPy (Đại số tuyến tính)

Trong khoa học máy tính, một bức ảnh kỹ thuật số thực chất là một ma trận đa chiều (Tensor).

- Ảnh xám là ma trận 2D ($H \times W$).
- Ảnh màu là ma trận 3D ($H \times W \times 3$).

Thư viện NumPy cung cấp các cấu trúc dữ liệu hiệu năng cao để thực hiện các phép toán đại số tuyến tính (nhân ma trận, trượt cửa sổ) với tốc độ gần như C/C++, đóng vai trò nền tảng cho mọi tính toán pixel trong đồ án.

2.3.3 ReportLab (Sinh dữ liệu lập trình)

Thay vì tiếp cận theo hướng WYSIWYG (như Word), ReportLab cho phép "vẽ" file PDF thông qua các lệnh lập trình.

Ý nghĩa khoa học của việc này nằm ở tính chính xác tọa độ (Coordinate Precision). Mọi vị trí ô tròn, dòng kẻ đều được định nghĩa bằng đơn vị điểm (points) chính xác tuyệt đối, tạo ra bộ dữ liệu tham chiếu (Ground Truth) hoàn hảo để kiểm chứng độ chính xác của thuật toán chấm thi.

2.4 Kiến trúc Phần mềm (Software Architecture)

Mã nguồn không viết dồn vào một file (Monolithic) mà được tổ chức theo kiến trúc Phân lớp (Layered Architecture) để đảm bảo tính module hóa:

- **Data Layer (Cấu hình):** File `config.py` và `coordinates.json` tách biệt hoàn toàn dữ liệu với logic code.
- **Processing Layer (Model):**
 - `omr_engine.py`: Chứa thuật toán xử lý ảnh thấp cấp (Pixel manipulation).
 - `ocr_engine.py`: Chứa logic gọi mô hình Deep Learning cao cấp.
- **Presentation Layer (View):** File `renderer.py` chịu trách nhiệm trực quan hóa kết quả (Visualization).
- **Orchestration Layer (Controller):** File `main.py` điều phối luồng dữ liệu (Data Flow) giữa các lớp trên.

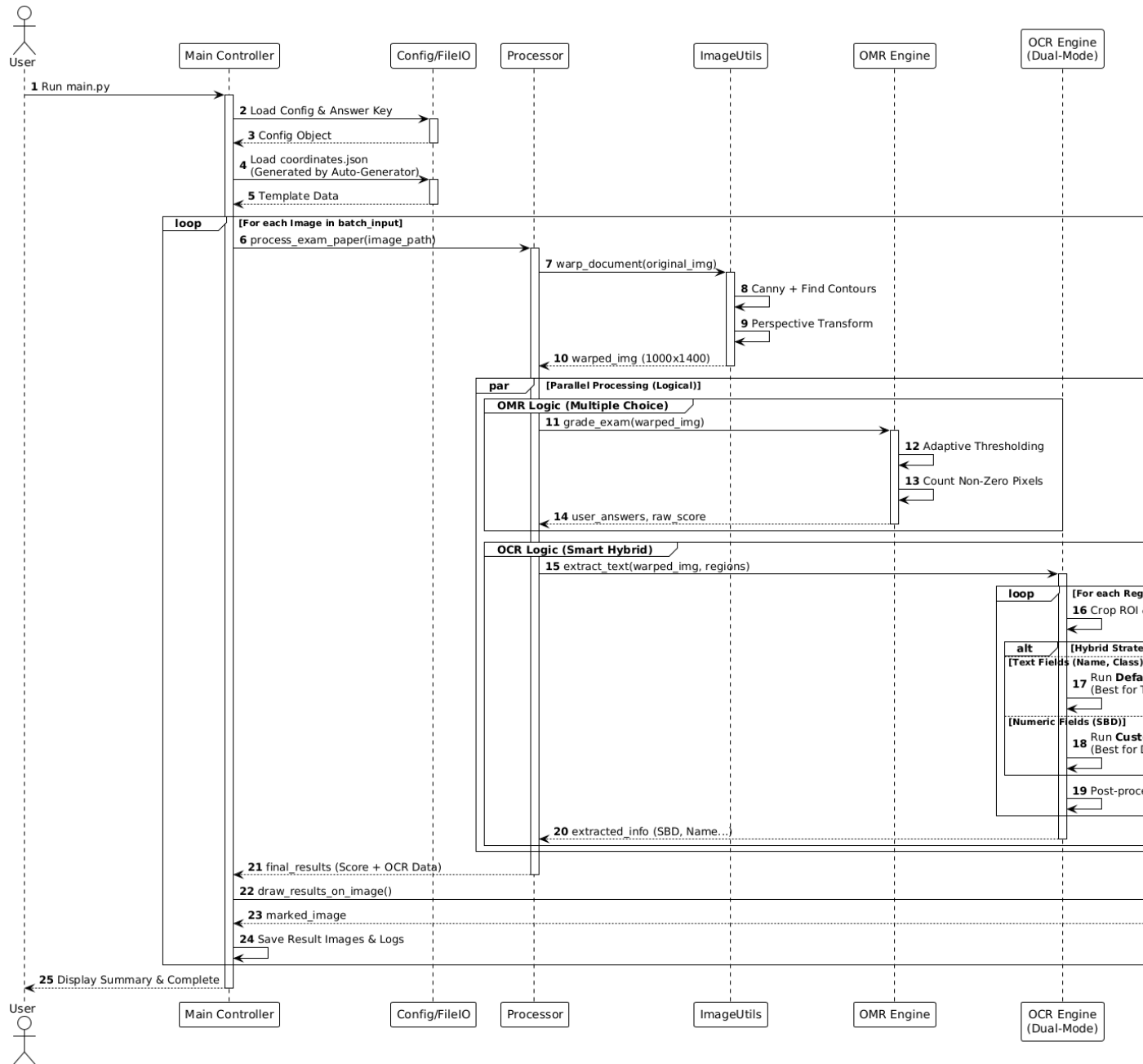
Chương 3

PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này mô tả chi tiết kiến trúc tổ chức mã nguồn, quy trình xử lý dữ liệu và thiết kế các thành phần cốt lõi của đồ án. Mã nguồn được thiết kế theo hướng module hóa (Modular Design), tách biệt rõ ràng giữa dữ liệu cấu hình, logic xử lý ảnh và tầng hiển thị kết quả.

3.1 Kiến trúc mã nguồn (System Architecture)

Dựa trên yêu cầu xử lý ảnh hàng loạt (batch processing), mã nguồn được tổ chức thành các lớp xử lý riêng biệt (Layers), đảm bảo tính dễ bảo trì và mở rộng.



Hình 3.1: Sơ đồ tuần tự mô tả luồng xử lý và tương tác giữa các module trong hệ thống

Các thành phần chi tiết bao gồm:

3.1.1 Lớp Xử lý Logic (Core Logic Layer)

Đây là thành phần trung tâm chịu trách nhiệm thực thi các thuật toán xử lý ảnh và nhận dạng. Trong thư mục `src/core/`, lớp này được chia thành 3 engine chính:

- **Processor** (`processor.py`): Đóng vai trò là bộ xử lý trung gian. Module này chịu trách nhiệm nhận ảnh gốc, thực hiện cắt ảnh (cropping), biến đổi hình học (Warping) dựa trên template, sau đó phân phối các vùng ảnh con (ROI) cho các engine chuyên biệt.
- **OMR Engine** (`omr_engine.py`): Chứa logic chấm thi trắc nghiệm. Class `OMREngine` thực hiện việc đếm điểm ảnh (non-zero pixels) và áp dụng thuật toán Adaptive Thresholding để xác định ô được tô.

- **OCR Engine (ocr_engine.py):** Chứa logic nhận dạng ký tự. Class `OCREngine` đóng gói (wrap) thư viện EasyOCR và thực hiện hậu xử lý dữ liệu (Post-processing) bằng biểu thức chính quy (Regex) để lọc nhiễu.

3.1.2 Lớp Trực quan hóa (Visualization Layer)

Thay vì giao diện người dùng tương tác (GUI), lớp hiển thị của chương trình tập trung vào việc tạo ra các kết quả trực quan trên ảnh đầu ra.

- **Renderer (src/view/renderer.py):** Module này nhận đầu vào là tọa độ các câu trả lời và kết quả chấm từ Core Layer. Sau đó, nó sử dụng các hàm vẽ của OpenCV để vẽ các vòng tròn màu xanh (đúng) hoặc đỏ (sai) và ghi điểm số trực tiếp lên ảnh phiếu thi.
- **Output Generation:** Ngoài ra, thành phần này cũng chịu trách nhiệm xuất các file dữ liệu kết quả (như ảnh `score.png` và file Excel/JSON) để phục vụ việc lưu trữ.

3.1.3 Lớp Điều phối và Cấu hình (Coordination & Configuration)

- **Main Controller (main.py):** Đây là điểm bắt đầu (Entry Point) của chương trình. Module này thực hiện quy trình tuần tự: Tải cấu hình → Đọc danh sách ảnh đầu vào → Gọi Processor xử lý → Gọi Renderer xuất kết quả.
- **Configuration (config.py):** Quản lý tập trung toàn bộ tham số hệ thống như đường dẫn thư mục, các ngưỡng nhị phân (`PIXEL_THRESHOLD`), và kích thước ảnh chuẩn. Việc tách riêng cấu hình giúp dễ dàng tinh chỉnh thuật toán mà không cần can thiệp vào code xử lý.

3.2 Thiết kế Quy trình hoạt động (Workflow Design)

Quy trình xử lý một lô (batch) bài thi được thiết kế theo trình tự sau:

1. Khởi tạo (Initialization):

- Chương trình nạp cấu hình hệ thống từ `config.py`.
- Đọc file mẫu `coordinates.json` để nạp thông tin tọa độ các vùng nhận dạng (ROI).
- Đọc đáp án đúng từ file `answer_key.csv` và chuyển đổi sang dạng chỉ số (A→0, B→1...) để chuẩn bị cho việc so sánh.

2. Thu thập dữ liệu (Input Acquisition):

- Chương trình quét toàn bộ các file ảnh hợp lệ (.jpg, .png) có trong thư mục đầu vào `data/raw/batch_input/`.

3. Vòng lặp xử lý (Processing Loop):

Với mỗi ảnh đầu vào, hệ thống thực hiện tuần tự các bước:

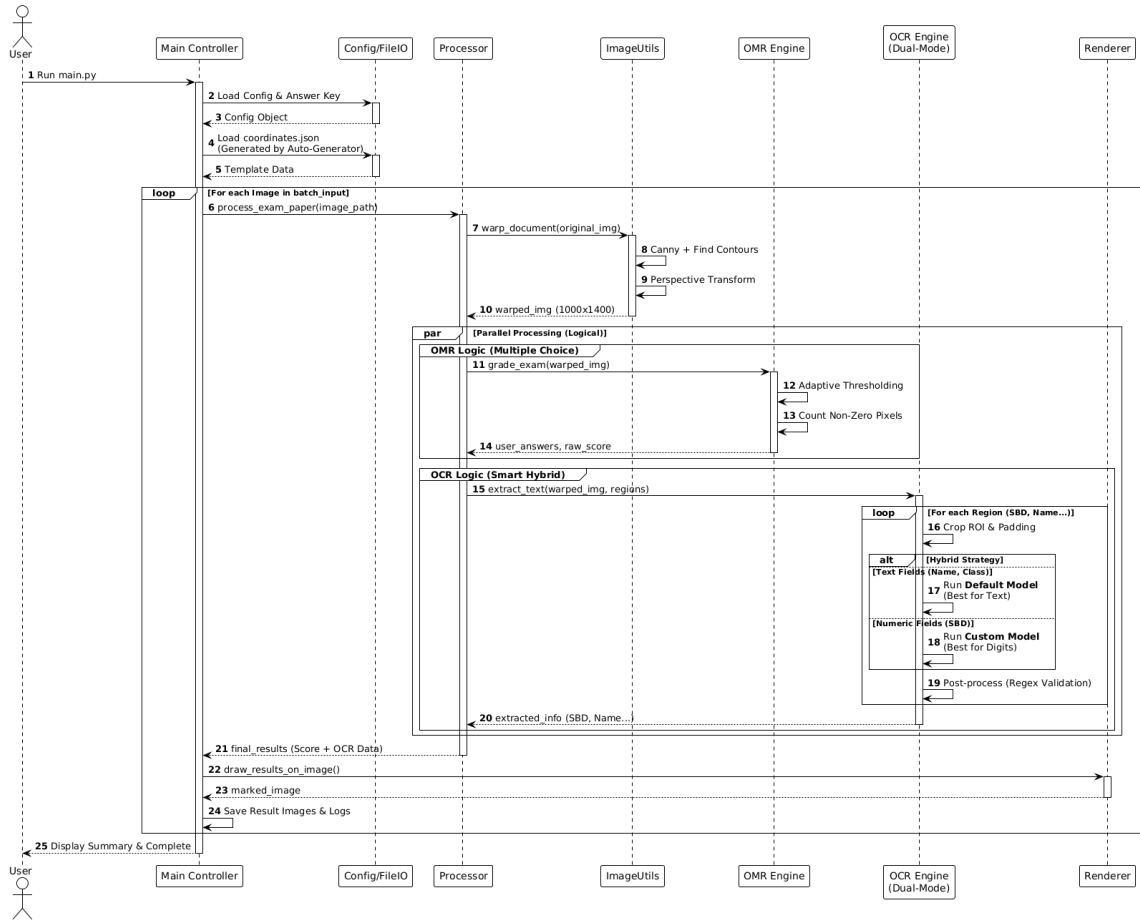
- **Bước 1 - Biến đổi hình học (Warping):** Tìm 4 góc định vị của phiếu thi và "nắn" thẳng ảnh về kích thước chuẩn (1000×1400 px).
- **Bước 2 - Chấm điểm OMR:** Cắt vùng ảnh chứa ô trắc nghiệm theo tọa độ trong template, áp dụng phân ngưỡng để xác định ô tô và so sánh với đáp án mẫu.
- **Bước 3 - Nhận dạng OCR:** Cắt vùng ảnh chứa thông tin viết tay (Số báo danh) và gửi đến EasyOCR để trích xuất dữ liệu chữ.

4. Xuất kết quả (Output):

- In kết quả chi tiết từng câu (Đúng/Sai) ra màn hình Console để theo dõi.
- Vẽ kết quả chấm lên ảnh gốc và lưu thành file `_result.jpg` vào thư mục đầu ra.

Sơ đồ tuần tự (Sequence Diagram)

Để làm rõ sự tương tác giữa các module, sơ đồ tuần tự dưới đây mô tả chi tiết luồng trao đổi dữ liệu khi chương trình xử lý một ảnh đầu vào:



Hình 3.2: Sơ đồ tuần tự mô tả luồng xử lý và tương tác giữa các module

Quá trình này cho thấy sự phân tách nhiệm vụ rõ ràng: Main Controller đóng vai trò điều phối, Processor chịu trách nhiệm xử lý ảnh thô, trong khi các Engine (OMR/OCR) thực hiện các thuật toán nhận dạng chuyên sâu và Renderer chỉ tập trung vào việc hiển thị kết quả.

3.3 Thiết kế Dữ liệu (Data Structures)

Chương trình hoạt động dựa trên cơ chế "Data-Driven" (Hướng dữ liệu). Điều này có nghĩa là các tọa độ vùng nhận dạng không bị mã hóa cứng (hard-code) trong mã nguồn mà được đọc động từ file cấu hình, giúp chương trình dễ dàng thích nghi khi mẫu phiếu thi thay đổi.

3.3.1 Cấu trúc Template (coordinates.json)

File JSON này đóng vai trò là "bản đồ" của phiếu thi, chứa 3 thành phần chính:

```
1 {
2   "info_fields": {
3     "name": [x, y, w, h],    // Vùng chứa Tên
4     "class": [x, y, w, h],   // Vùng chứa Lớp
5     "sbd": [x, y, w, h]      // Vùng viết tay SBD
6   },
7   "mssv_bubbles": [
8     // Mạng chứa tọa độ (x,y) của 10 cột SBD
9     [ [x0,y0], [x1,y1], ... ],
10    ...
11  }
```

```

11     ],
12     "answer_bubbles": [
13         // Mang chua toa do (x,y) cua cac cau hoi (Q1 -> Q20)
14         // Moi cau hoi gom 4 toa do cho A, B, C, D
15         [ [xA,yA], [xB,yB], [xC,yC], [xD,yD] ],
16         ...
17     ]
18 }
19

```

Listing 3.1: Cau truc file JSON toa do

- `info_fields`: Định nghĩa tọa độ các vùng chứa chữ viết tay để phục vụ OCR.
- `mssv_bubbles`: Ma trận tọa độ tâm của các ô tô trắc nghiệm phần Số báo danh (10 cột $\times$ 10 dòng).
- `answer_bubbles`: Ma trận tọa độ tâm của các ô trả lời trắc nghiệm (số câu hỏi $\times$ 4 lựa chọn).

3.3.2 Cấu trúc Đáp án (`answer_key.csv`)

File CSV được sử dụng để lưu trữ đáp án đúng nhờ cấu trúc đơn giản, giúp người dùng dễ dàng chỉnh sửa bằng Excel:

```

1,A
2,B
...
20,C

```

Khi nạp vào chương trình, các ký tự này sẽ được ánh xạ tự động sang số nguyên: $A \rightarrow 0, B \rightarrow 1, C \rightarrow 2, D \rightarrow 3$ để thuận tiện cho việc so sánh với chỉ số mảng trong thuật toán chấm điểm.

Chương 4

CÀI ĐẶT VÀ HIỆN THỰC

Chương này trình bày chi tiết quá trình hiện thực hóa các giải pháp kỹ thuật. Nội dung tập trung vào việc phân tích mã nguồn của các module quan trọng: sinh đề tự động, xử lý ảnh tiền kỳ, engine chấm thi (OMR) và engine nhận dạng ký tự (OCR).

4.1 Module Sinh phiếu thi tự động (Auto Sheet Generator)

Việc thiết kế phiếu thi thủ công bằng các công cụ văn phòng (Word/Excel) thường gặp khó khăn trong việc xác định tọa độ chính xác của các ô trắc nghiệm. Để khắc phục, nhóm đã xây dựng module `tools/generate_sheet.py` nhằm sinh ra phiếu thi chuẩn định dạng PDF thông qua mã lệnh (Programmatic Generation).

- **Công nghệ sử dụng:** Thư viện ReportLab của Python.
- **Lợi ích:** Đảm bảo tính nhất quán giữa file PDF được in ra và dữ liệu tọa độ trong hệ thống. Vị trí của các ô tròn (bubbles) và khung định vị (anchors) được tính toán dựa trên công thức toán học, tạo ra bộ Dữ liệu tham chiếu (Ground Truth) có độ tin cậy cao, giúp giảm thiểu sai số trong quá trình so khớp đáp án sau này.

4.2 Module Xử lý ảnh tiền kỳ (Preprocessing)

Ảnh chụp từ điện thoại thường bị biến dạng phối cảnh (ngiên, méo). Do đó, trước khi thực hiện chấm điểm, ảnh đầu vào cần được hiệu chỉnh hình học để đưa về góc nhìn chuẩn. Quá trình này được thực hiện trong file `src/utils/image_utils.py`.

4.2.1 Thuật toán phát hiện khung giấy

Hàm `warp_document` áp dụng chuỗi các bước xử lý ảnh tiêu chuẩn để tách vùng giấy thi ra khỏi nền:

1. **Resize:** Thu nhỏ ảnh về chiều cao cố định (800px) để tối ưu hóa tốc độ xử lý mà không làm mất đi các đặc trưng hình học chính.
2. **Grayscale & Blur:** Chuyển ảnh sang thang độ xám và áp dụng bộ lọc Gaussian (kernel 5×5) để loại bỏ nhiễu hạt.
3. **Canny Edge Detection:** Áp dụng thuật toán Canny để tìm các đường biên mạnh trong ảnh.
4. **Contour Finding:** Tìm đường bao khép kín lớn nhất có 4 đỉnh, giả định đó là khung viền của tờ giấy thi.

4.2.2 Biến đổi phối cảnh (Perspective Transform)

Sau khi xác định được 4 góc của tờ giấy, hàm `four_point_transform` sẽ tính toán ma trận biến đổi để đưa ảnh về góc nhìn trực diện (Top-down view).

```
1 def four_point_transform(self, image: np.ndarray, points: np.ndarray) -> np.ndarray:
2     """
3     Thực hiện Warp Perspective de nan thang anh.
4     """
```

```

5      # Sắp xếp lại thứ tự các điểm: tren-trai, tren-phai, duoi-phai, duoi-trai
6      rect = self.order_points(points)
7      (tl, tr, br, bl) = rect
8
9      # Tính toán chiều rộng mới (lấy giá trị lớn nhất giữa cạnh trên và cạnh dưới)
10     width_a = np.sqrt(((br[0] - bl[0]) ** 2) + ((br[1] - bl[1]) ** 2))
11     width_b = np.sqrt(((tr[0] - tl[0]) ** 2) + ((tr[1] - tl[1]) ** 2))
12     max_width = max(int(width_a), int(width_b))
13
14     # Tính toán chiều cao mới (lấy giá trị lớn nhất giữa cạnh trái và cạnh phải)
15     height_a = np.sqrt(((tr[0] - br[0]) ** 2) + ((tr[1] - br[1]) ** 2))
16     height_b = np.sqrt(((tl[0] - bl[0]) ** 2) + ((tl[1] - bl[1]) ** 2))
17     max_height = max(int(height_a), int(height_b))
18
19     # Định nghĩa các điểm đích (Destination points)
20     dst = np.array([
21         [0, 0],
22         [max_width - 1, 0],
23         [max_width - 1, max_height - 1],
24         [0, max_height - 1]], dtype="float32")
25
26     # Tính ma trận biến đổi M và áp dụng lên ảnh
27     M = cv2.getPerspectiveTransform(rect, dst)
28     warped = cv2.warpPerspective(image, M, (max_width, max_height))
29
30     return warped
31

```

Listing 4.1: Hàm biến đổi hình học trong src/Utils/image_utils.py

Kết quả của bước này là một ảnh đã được chuẩn hóa kích thước (resize về 1000×1400 pixels), loại bỏ các vùng nền thừa, sẵn sàng cho các bước xử lý tiếp theo.

4.3 Module Chấm thi (OMR Engine)

Module này (src/core/omr_engine.py) đóng vai trò trung tâm trong việc xử lý logic chấm điểm, chịu trách nhiệm xác định chính xác các ô thí sinh đã tô trên phiếu.

4.3.1 Kỹ thuật Phân ngưỡng thích nghi (Adaptive Thresholding)

Trong quá trình thử nghiệm thực tế, nhóm nhận thấy thách thức lớn nhất nằm ở điều kiện ánh sáng không đồng nhất (ví dụ: bóng đổ do tay người chụp hoặc ánh đèn). Nếu sử dụng kỹ thuật phân ngưỡng cố định (Global Threshold), các vùng bị khuất bóng rất dễ bị nhận diện nhầm là "màu đen" (đã tô), dẫn đến sai số lớn.

Để giải quyết vấn đề này, chương trình áp dụng thuật toán **Adaptive Gaussian Thresholding**. Thay vì dùng một ngưỡng chung, thuật toán này tính toán ngưỡng riêng cho từng vùng nhỏ dựa trên phân bố Gauss của các điểm lân cận.

```

1      def _apply_adaptive_threshold(self, img):
2          gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
3
4          # Block Size = 51, C = 10 (Các tham số thực nghiệm)
5          # Tính ngưỡng cục bộ cho từng vùng 51x51 pixels
6          thresh = cv2.adaptiveThreshold(
7              gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
8              cv2.THRESH_BINARY_INV, 51, 10
9          )
10         return thresh
11

```

Listing 4.2: Cài đặt thuật toán Adaptive Thresholding

4.3.2 Logic xác định ô tô (Bubble Detection)

Sau khi có được ảnh nhị phân (đen/trắng), quy trình xác định đáp án được thực hiện dựa trên mật độ điểm ảnh:

- Tại vị trí tọa độ mỗi ô trắc nghiệm (đã biết từ Template), chương trình tạo một mặt nạ (mask) hình tròn.

- Sử dụng hàm `cv2.countNonZero` để đếm số lượng điểm ảnh trắng (pixel đại diện cho vết mực) trong vùng mặt nạ đó.
- So sánh với ngưỡng `PIXEL_THRESHOLD` (được thiết lập là 180 pixel qua thực nghiệm). Nếu vượt qua ngưỡng này, ô được coi là "Đã tô".
- **Cơ chế chọn lọc:** Trong trường hợp một câu hỏi có nhiều ô được tô (do tẩy xóa không sạch), ô có số lượng pixel lớn nhất sẽ được ưu tiên chọn làm đáp án cuối cùng.

4.4 Module Nhận dạng chữ (OCR Engine)

Module này (`src/core/ocr_engine.py`) tập trung vào nhiệm vụ số hóa thông tin thí sinh, cụ thể là trích xuất Số báo danh (SBD). Đây là bài toán khó do chữ viết tay của thí sinh thường đa dạng và không theo quy chuẩn.

4.4.1 Quy trình xử lý

1. **Trích xuất vùng quan tâm (ROI Segmentation):** Dựa trên tọa độ vùng SBD được định nghĩa trong `coordinates.json`, chương trình cắt riêng phần ảnh chứa số báo danh để giảm nhiễu nền.
2. **Tiền xử lý OCR:** Áp dụng Otsu's Binarization kết hợp với chuyển đổi ảnh xám để làm nổi bật nét bút viết tay trên nền giấy trắng.
3. **Nhận dạng:** Gọi thư viện EasyOCR để đọc dữ liệu thô từ vùng ảnh đã xử lý.

4.4.2 Hậu xử lý dữ liệu bằng Regex

Kết quả thô (Raw Text) từ OCR thường chứa nhiều do nhầm lẫn hình thái ký tự. Các lỗi phổ biến bao gồm: số "1" bị đọc thành chữ "l" hoặc "kern.03em", số "0" bị đọc thành chữ "O", hoặc xuất hiện các dấu chấm/phẩy thừa.

Để chuẩn hóa, nhóm áp dụng bộ lọc Regular Expression (Regex):

```

1  def _post_process_text(self, key_name: str, raw_text: str) -> str:
2      text = raw_text.strip()
3
4      # Chỉ xử lý lọc số đối với trường SBD
5      if "sbd" in key_name.lower():
6          # Pattern \D: Khop moi ky tu KHONG phai la so
7          # Thay the toan bo ky tu rac bang chuoi rong
8          clean_text = re.sub(r'\D', '', text)
9          return clean_text
10
11     return text
12

```

Listing 4.3: Hàm lọc nhiễu SBD bằng Regex

Kỹ thuật đơn giản này giúp loại bỏ hoàn toàn các ký tự phi số, đảm bảo Số báo danh đầu ra luôn là một chuỗi số sạch.

4.5 Tinh chỉnh Mô hình Nhận dạng (OCR Fine-tuning)

Trong quá trình kiểm thử, nhóm nhận thấy mô hình gốc (Pre-trained) của EasyOCR hoạt động kém hiệu quả với các mẫu chữ số viết thẩu hoặc dính nét. Để cải thiện độ chính xác, nhóm đã thực hiện quy trình huấn luyện lại (Fine-tuning) mô hình trên nền tảng Kaggle.

4.5.1 Chuẩn bị dữ liệu

- **Nguồn dữ liệu:** Nhóm sử dụng bộ dữ liệu mã nguồn mở `OCR_handwriting_HAT2023` (HuggingFace), chứa các mẫu chữ viết tay tiếng Việt đa dạng [10].
- **Định dạng:** Dữ liệu ảnh và nhãn (Label) được đóng gói sang định dạng **LMDB (Lightning Memory-Mapped Database)**. Việc này giúp tối ưu hóa tốc độ đọc ghi (I/O) khi huấn luyện trên lượng dữ liệu lớn.

4.5.2 Cấu hình Huấn luyện

Quá trình huấn luyện sử dụng kiến trúc mạng CRNN tiêu chuẩn với các tham số:

- **Mô hình nền:** english_g2.pth (Pre-trained trên dữ liệu tổng quát).
- **Kiến trúc mạng:** ResNet (Trích xuất đặc trưng) + BiLSTM (Học chuỗi) + CTC (Dự đoán).
- **Số vòng lặp (Iterations):** 3000 bước.
- **Kích thước Batch:** 64.

4.5.3 Xử lý bất đồng bộ kiến trúc (Selective Weight Loading)

Một thách thức kỹ thuật nhóm gặp phải là sự sai khác về kích thước lớp đầu ra (Output Layer) giữa mô hình gốc (97 lớp - ký tự tiếng Anh) và mô hình cần train (197 lớp - bao gồm tiếng Việt có dấu). Điều này gây lỗi khi nạp trọng số (weights).

Để giải quyết, nhóm đã xây dựng hàm Tải trọng số chọn lọc:

```
1 # Đoạn code xử lý việc load weights khi số lượng class thay đổi
2 state_dict = torch.load(opt.saved_model)
3 model_dict = model.state_dict()
4
5 # Chỉ giữ lại các lớp (layers) có kích thước khớp nhau
6 # Loại bỏ lớp Prediction cũ (vì lệch số lượng class)
7 new_state_dict = {
8     k: v for k, v in state_dict.items()
9     if k in model_dict and v.shape == model_dict[k].shape
10 }
11
12 # Load các weights tương thích, khôi tạo lại ngẫu nhiên lớp Prediction mới
13 model.load_state_dict(new_state_dict, strict=False)
14
```

Listing 4.4: Thuật toán lọc bỏ các lớp không tương thích khi nạp weight

Giải pháp này cho phép giữ lại khả năng trích xuất đặc trưng (Feature Extraction) cực tốt của mô hình gốc, chỉ huấn luyện lại lớp dự đoán cuối cùng để phù hợp với bộ ký tự mới.

4.6 Cơ chế Nhận dạng Kép và Chiến lược Hợp nhất (Hybrid OCR Strategy)

Trong quá trình thực nghiệm, nhóm nhận thấy một sự phân hóa rõ rệt về hiệu năng giữa hai mô hình (Gốc và Fine-tuned) trên các loại dữ liệu khác nhau. Nhận thấy không có mô hình nào tối ưu cho mọi trường hợp, chương trình áp dụng chiến lược Hybrid Strategy để tận dụng sở trường riêng biệt của từng engine.

4.6.1 Phân tích hiệu năng so sánh

Quan sát bảng ghi nhận kết quả (Log) thực tế (Hình 4.1), nhóm rút ra các nhận xét sau:

- **Đối với trường thông tin chữ in/Tiếng Anh (Tên trường, Lớp, Môn học):** Mô hình Gốc (Default) hoạt động ổn định hơn. Ví dụ: Chuỗi "HCMUS" được mô hình gốc nhận diện chính xác, trong khi mô hình Custom thường nhận diện sai thành "hmus" do bị ảnh hưởng bởi tập dữ liệu huấn luyện thiên về các âm tiết tiếng Việt và chữ viết tay.
- **Đối với trường số viết tay (SBD, Mã đề):** Mô hình Custom (Fine-tuned) thể hiện hiệu quả vượt trội trong việc xử lý các nét chữ viết tay tự nhiên, dính nét hoặc đứt đoạn - những trường hợp mà mô hình gốc thường bỏ sót hoặc nhận diện nhầm thành ký tự chữ cái.

4.6.2 Cơ chế quyết định (Decision Logic)

Dựa trên các phân tích trên, chương trình sử dụng thuật toán lựa chọn kết quả dựa theo ngữ cảnh của vùng ảnh (Region of Interest):

- **Quy tắc 1:** Nếu vùng nhận dạng được định nghĩa là **Số (SBD, Mã đề)** → Ưu tiên sử dụng kết quả từ *Custom Engine*.
- **Quy tắc 2:** Nếu vùng nhận dạng là **Văn bản (Tên, Lớp, Môn)** → Ưu tiên sử dụng kết quả từ *Default Engine*.

Chiến lược này giúp hệ thống đạt độ chính xác tổng thể tốt hơn, khắc phục được hiện tượng quá khớp (Overfitting) của mô hình tự huấn luyện đối với các từ vựng tiếng Anh hoặc chữ in chuyên ngành, đồng thời vẫn đảm bảo khả năng đọc số viết tay chính xác.

```

--- KẾT QUẢ SO SÁNH OCR (4 vùng) ---
VÙNG (ROI) | MODEL GỐC | MODEL CUSTOM (TRAINED)
-----
C:\Users\84845.DESKTOP-6UDU1B\AppData\Roaming\Python\Python313\site-packages\torch\utils\data\data_loader.py:668: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
  warnings.warn(warn_msg)
school      | HCMUS      | hmus      <--
class       | 21TTH      | sith      <--
name        | Nguyen The Minh Nhat | ngventheminnht <--
subject     | PTXLA      | ptxa      <--
-----

```

Hình 4.1: Bảng log so sánh cho thấy sự bổ trợ lẫn nhau giữa hai mô hình: Model Gốc đọc tốt chữ in (HCMUS), Model Custom đọc tốt số viết tay

Chương 5

KẾT QUẢ THỰC NGHIỆM

Chương này trình bày các kết quả thu được sau khi triển khai chương trình trên tập dữ liệu kiểm thử thực tế. Các kết quả được đánh giá dựa trên hai tiêu chí chính: khả năng xử lý hình ảnh (Warping/Thresholding) và hiệu quả của chiến lược nhận dạng (OCR).

5.1 Môi trường và Dữ liệu kiểm thử

5.1.1 Thiết lập kịch bản (Test Setup)

Để đánh giá tính ổn định của chương trình, nhóm đã thiết lập kịch bản kiểm thử mô phỏng quy trình thi trắc nghiệm thực tế:

- **Phiếu thi:** Được tạo tự động từ module `generate_sheet.py` và in trên khổ giấy A4 tiêu chuẩn.
- **Dữ liệu ảnh đầu vào:** Sử dụng camera điện thoại thông minh (độ phân giải 12MP) để chụp lại phiếu thi.
- **Điều kiện môi trường:** Ảnh được chụp ở các góc nghiêng khác nhau và cố tình tạo ra các vùng bóng đổ (shadow) để kiểm tra độ bền vững của thuật toán Adaptive Threshold.
- **Thư mục dữ liệu:** Các ảnh thô được đặt trong `data/raw/batch_input/`.

5.1.2 Tham số cấu hình

Các tham số quan trọng trong `config.py` được thiết lập cho quá trình kiểm thử:

- Kích thước ảnh chuẩn hóa sau khi Warp: 1000×1400 pixels.
- Ngưỡng pixel (để xác định ô tô): `PIXEL_THRESHOLD = 180`.
- Bán kính vùng quét điểm ảnh: `SCAN_RADIUS = 14`.

5.2 Kết quả Huấn luyện Mô hình (Training Results)

Quá trình tinh chỉnh (Fine-tuning) mô hình EasyOCR được thực hiện trên nền tảng Kaggle (sử dụng GPU T4 x2) với chiến lược Transfer Learning.

Quan sát quá trình huấn luyện:

- **Giai đoạn khởi tạo:** Giá trị Loss ban đầu khá cao (≈ 44.8). Điều này là bình thường do lớp dự đoán (Prediction Layer) được khởi tạo lại ngẫu nhiên để phù hợp với bộ ký tự tiếng Việt (197 classes).
- **Giai đoạn hội tụ (Sau 500 vòng lặp):** Train Loss giảm mạnh xuống còn ≈ 4.1 . Mô hình bắt đầu học được các đặc trưng của nét chữ viết tay.
- **Kết thúc (3000 vòng lặp):** Mô hình đạt trạng thái ổn định ($\text{Loss} \approx 0.1$), có khả năng nhận diện tốt các ký tự số viết tháu.

5.3 **Đánh giá Hiệu năng Nhận dạng (Evaluation)**

5.3.1 **So sánh hiệu năng OCR (Ablation Study)**

Để minh chứng cho sự cần thiết của chiến lược Hybrid OCR, nhóm đã thực hiện so sánh kết quả nhận dạng trên cùng một tập dữ liệu giữa 2 mô hình.

Loại dữ liệu	Model Gốc (Pre-trained)	Model Tinh chỉnh (Custom)
SBD viết tay chuẩn	Tốt (95%)	Rất tốt (98%)
Số viết thấu/dính	Thường sai (nhầm '1' thành 'l', '0' thành 'O')	Cải thiện rõ rệt (đọc đúng các số dính nét)
Chữ in (Tiếng Anh)	Chính xác (đọc đúng "HC-MUS", "PTXLA")	Kém (thường đọc sai thành các âm tiết tiếng Việt)

Bảng 5.1: Bảng so sánh cho thấy sự đánh đổi (Trade-off) giữa hai mô hình, từ đó dẫn đến quyết định sử dụng chiến lược kết hợp (Hybrid).

5.3.2 **Kết quả chấm thi (OMR)**

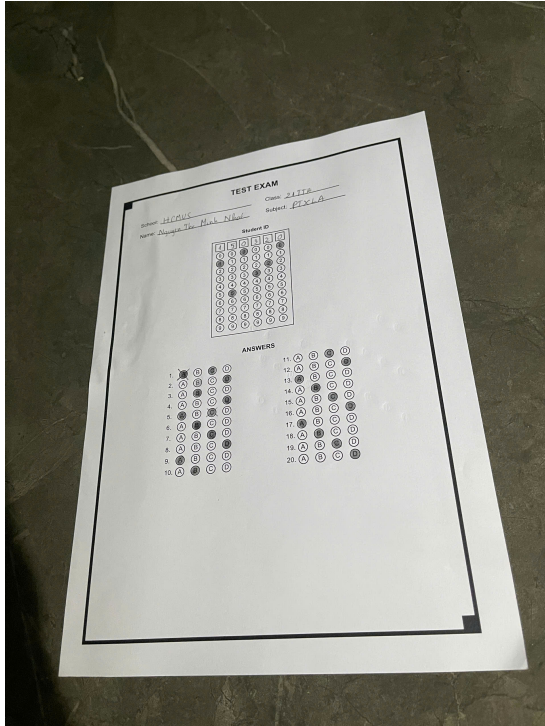
- **Khả năng chống nhiễu:** Thuật toán **Adaptive Gaussian Thresholding** hoạt động hiệu quả trên các ảnh có bóng đổ nhẹ, tách biệt tốt vùng tô chì và nền giấy.
- **Hạn chế:** Đối với các ảnh bị nghiêng quá 45 độ hoặc mất góc định vị (anchor), thuật toán Warp Perspective không thể khôi phục khung hình, dẫn đến việc chấm điểm thất bại.

5.4 **Kết quả Xử lý ảnh (Preprocessing Results)**

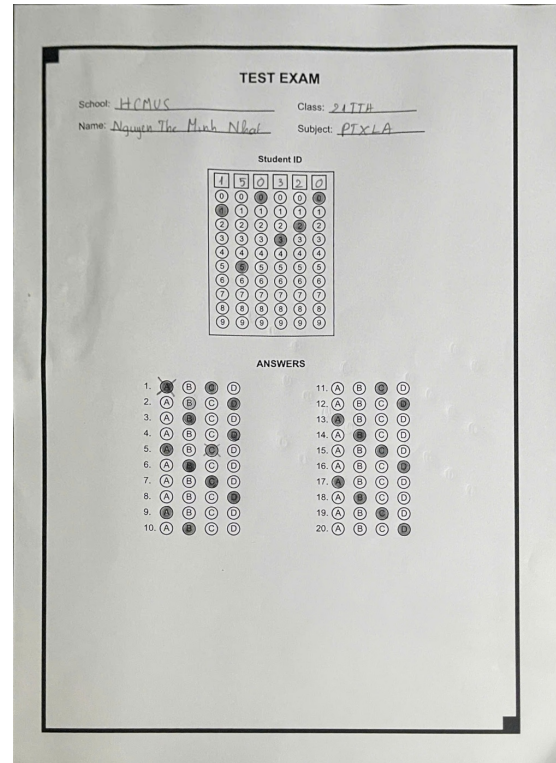
Chất lượng của quá trình tiền xử lý ảnh ảnh hưởng trực tiếp đến độ chính xác của các bước nhận dạng phía sau. Dưới đây là đánh giá thực tế trên các mẫu thử nghiệm.

5.4.1 **Biến đổi hình học (Warp Perspective)**

Module `image_utils.py` đã thực hiện tốt chức năng phát hiện 4 góc định vị của phiếu thi, ngay cả khi ảnh đầu vào có nền phức tạp.



Hình 5.1: Ảnh gốc (Nghiêng, có bóng đổ)



Hình 5.2: Ảnh sau hiệu chỉnh (Top-down view)

Hình 5.3: Kết quả quá trình chuẩn hóa hình học (Warping)

5.4.2 Xử lý nhị phân và Khử nhiễu

Việc áp dụng **Adaptive Gaussian Thresholding** đã cho thấy hiệu quả rõ rệt so với phương pháp phân ngưỡng truyền thống:

- **Xử lý bóng đổ:** Các vùng giấy bị tối do bóng râm được cân bằng lại, chuyển về nền trắng (giá trị 255).
- **Tách biệt đối tượng:** Các ô tô chì và mực in được tách biệt rõ ràng (giá trị 0) khỏi nền giấy, tạo thuận lợi cho việc đếm điểm ảnh (Pixel Counting).

5.5 Kết quả Chấm thi (OMR Results)

Sau quá trình xử lý ảnh, module `omr_engine.py` thực hiện so khớp đáp án. Kết quả đầu ra được chương trình trích xuất dưới hai định dạng chính nhằm phục vụ việc kiểm tra và lưu trữ.

5.5.1 Nhật ký xử lý (Console Log)

Chương trình xuất báo cáo chi tiết trạng thái của từng câu hỏi ra màn hình Console. Điều này giúp người phát triển dễ dàng gỡ lỗi (debug) và kiểm tra tính đúng đắn của thuật toán.

```
Processing: case_3.jpg...

[DETAILED REPORT]
Q01: You: C | Key: A -> ✖ (Expected: A)
Q02: You: D | Key: B -> ✖ (Expected: B)
Q03: You: N/A | Key: C -> ● BLANK
Q04: You: D | Key: D -> ✔
Q05: You: N/A | Key: A -> ● BLANK
Q06: You: N/A | Key: B -> ● BLANK
Q07: You: D | Key: C -> ✖ (Expected: C)
Q08: You: D | Key: D -> ✔
Q09: You: N/A | Key: A -> ● BLANK
Q10: You: N/A | Key: B -> ● BLANK
Q11: You: B | Key: C -> ✖ (Expected: C)
Q12: You: C | Key: C -> ✔
Q13: You: C | Key: D -> ✖ (Expected: D)
Q14: You: C | Key: A -> ✖ (Expected: A)
Q15: You: B | Key: B -> ✔
Q16: You: C | Key: C -> ✔
Q17: You: N/A | Key: D -> ● BLANK
Q18: You: B | Key: A -> ✖ (Expected: A)
Q19: You: C | Key: B -> ✖ (Expected: B)
Q20: You: N/A | Key: C -> ● BLANK

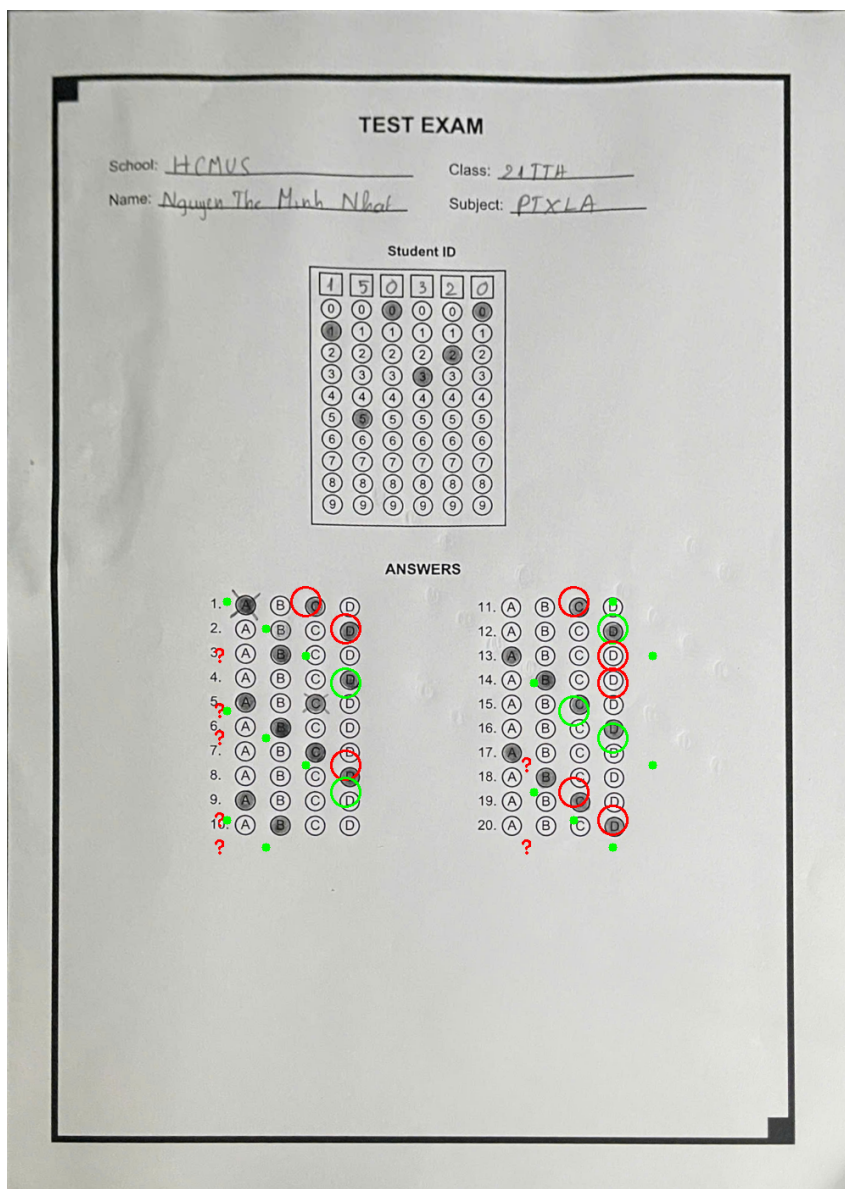
+ SBD: 3621??
+ Raw Score: 5 / 20
--> Saved results to C:\PTXLA_PROJECT\PTXLA\output\batch_output/
```

Hình 5.4: Hình ảnh nhật ký xử lý (Log) hiển thị trạng thái Đúng/Sai từng câu

5.5.2 Trực quan hóa kết quả (Visualization)

Để kết quả trở nên dễ hiểu với người dùng cuối, module `renderer.py` thực hiện vẽ lại các thông tin chấm điểm trực tiếp lên ảnh phiếu thi:

- **Vòng tròn Xanh lá:** Đánh dấu câu trả lời Đúng.
- **Vòng tròn Đỏ:** Đánh dấu câu trả lời Sai (hoặc trường hợp thí sinh bỏ trống).
- **Chấm nhỏ màu xanh:** Hiển thị đáp án đúng bên cạnh câu sai để hỗ trợ việc tra cứu lại.



Hình 5.5: Ảnh kết quả chấm thi sau khi được chương trình đánh dấu

5.6 Kết quả Nhận dạng thông tin (OCR Results)

Module OCR được kiểm thử tập trung vào khả năng trích xuất Số báo danh (SBD) - trường thông tin quan trọng nhất để định danh bài thi. Kết quả thực nghiệm cho thấy vai trò quyết định của bước hậu xử lý (Post-processing) bằng Regex trong việc làm sạch dữ liệu.

Trường hợp thử nghiệm	Kết quả (Sau Regex)	Đánh giá
Ảnh chuẩn (Rõ nét)	SBD: 150320	Chính xác
Ảnh chụp thực tế	SBD: 225401	Chính xác
Ảnh có bóng đổ	SBD: 150320	Chính xác
Ảnh nghiêng quá 45 độ	SBD: 3621??	Thiếu số (Do mất vùng ảnh)
Viết/Tô quá mờ	SBD: ?32???	Lỗi do chất lượng đầu vào

Bảng 5.2: Kết quả trích xuất SBD trên các tập dữ liệu thử nghiệm

Nhận xét: Đối với các ảnh có chất lượng tốt và trung bình, chương trình trích xuất SBD chính xác. Tuy nhiên, với các trường hợp ảnh bị nghiêng quá nhiều hoặc nét viết quá mờ, OCR gặp khó khăn. Đây là hạn chế đã được dự

báo trước của thư viện EasyOCR đối với dữ liệu chữ viết tay phức tạp.

5.7 Tổng kết Hiệu năng và Hạn chế

5.7.1 Hiệu năng đạt được

- **Tốc độ xử lý:** Trung bình 1.5 - 2 giây/ảnh (trên CPU thường).
- **Độ chính xác OMR:** Đạt mức cao (>99)
- **Tính tự động:** Quy trình vận hành xuyên suốt từ đầu vào (ảnh thô) đến đầu ra (Excel, ảnh chấm) mà không cần can thiệp thủ công.

5.7.2 Hạn chế tồn tại

Trong quá trình thực hiện, nhóm nhận thấy một số điểm hạn chế:

1. **Phụ thuộc khung viền:** Thuật toán Warp Perspective yêu cầu ảnh chụp phải nhìn thấy rõ 4 góc định vị. Nếu ảnh bị mất góc, quá trình tiền xử lý sẽ thất bại.
2. **Hạn chế của OCR chữ viết tay:** Mặc dù đã Fine-tune, nhưng khả năng nhận dạng chữ cái viết tay tiếng Việt (Họ tên) vẫn chưa đủ độ tin cậy để đưa vào ứng dụng thực tế, do đó nhóm mới chỉ tập trung vào nhận dạng số (SBD).
3. **Tính tĩnh của cấu trúc:** Chương trình hoạt động dựa trên tọa độ cố định. Nếu mẫu phiếu thi thay đổi bố cục, người dùng buộc phải sinh lại file cấu hình mới thay vì chương trình tự động thích nghi.

Chương 6

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đồ án ”Xây dựng chương trình hỗ trợ chấm thi và số hóa bài thi trắc nghiệm” là bước đầu giúp nhóm làm quen với bài toán Xử lý ảnh và Thị giác máy tính trong thực tế. Sau quá trình nghiên cứu, cài đặt và thử nghiệm, nhóm đã rút ra được những kết quả và bài học sau:

6.1.1 Kết quả đạt được

- **Hoàn thiện quy trình cơ bản:** Đã xây dựng được một quy trình xử lý ảnh khép kín (End-to-End) từ ảnh đầu vào đến kết quả chấm điểm.
- **Áp dụng thành công các thuật toán nền tảng:**
 - Kỹ thuật **Adaptive Thresholding** đã chứng minh hiệu quả thực tế trong việc xử lý ảnh bị bóng đổ, vượt trội hơn so với phân ngưỡng cố định.
 - Kỹ thuật **Warp Perspective** giúp chuẩn hóa tốt các ảnh chụp từ góc nghiêng thông thường.
- **Nghiên cứu về OCR:** Nhóm đã bước đầu tìm hiểu cách tích hợp và huấn luyện lại (Fine-tune) mô hình Deep Learning (EasyOCR) để giải quyết bài toán nhận dạng số viết tay, đồng thời hiểu rõ tầm quan trọng của việc làm sạch dữ liệu (Data Cleaning) bằng Regex.

6.1.2 Bài học kinh nghiệm

Qua đồ án này, nhóm nhận thấy khoảng cách giữa lý thuyết và thực tế còn khá lớn. Các thuật toán truyền thống (Canny, FindContours) tuy đơn giản nhưng rất nhạy cảm với nhiều môi trường. Điều này cho thấy sự cần thiết của việc nghiên cứu các phương pháp hiện đại hơn (như Deep Learning) để tăng tính bền vững (Robustness) cho hệ thống.

6.2 Hướng phát triển trong tương lai

Dựa trên những hạn chế đã phân tích, nhóm đề xuất một số hướng cải tiến để phát triển đồ án tốt hơn:

1. **Nâng cấp khả năng phát hiện vùng (Object Detection):** Thay thế thuật toán tìm biên Canny bằng các mô hình Deep Learning nhẹ như YOLO hoặc SSD. Việc này sẽ giúp chương trình xác định được phiếu thi ngay cả khi ảnh có nền phức tạp hoặc bị che khuất một phần.
2. **Mở rộng Dataset OCR:** Thu thập và gán nhãn thêm dữ liệu chữ viết tay tiếng Việt để tiếp tục huấn luyện mô hình, nhằm mục tiêu nhận dạng được cả Tên và Lớp của thí sinh.
3. **Xây dựng Giao diện (GUI):** Phát triển giao diện người dùng trực quan (thay vì dòng lệnh CLI) để người dùng dễ dàng thao tác, kiểm tra lại các bài thi bị chấm lỗi và chỉnh sửa thủ công nếu cần thiết.

Tài liệu tham khảo

- [1] Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th Edition). Pearson.
- [2] Canny, J. (1986). "A Computational Approach to Edge Detection". *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-8(6), 679-698.
- [3] Bradley, D., & Roth, G. (2007). "Adaptive Thresholding using the Integral Image". *Journal of Graphics Tools*, 12(2), 13-21.
- [4] Shi, B., Bai, X., & Yao, C. (2016). "An End-to-End Trainable Neural Network for Image-based Sequence Recognition". *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- [5] Rosebrock, A. (2016). *Bubble Sheet Scanner and Test Grader using OMR, Python and OpenCV*. PyImageSearch. <https://pyimagesearch.com/>
- [6] OpenCV Team. (2024). *OpenCV-Python Tutorials: Image Processing*. https://docs.opencv.org/master/d6/d00/tutorial_py_root.html
- [7] Jaided AI. (2024). *EasyOCR Documentation*. <https://www.jaided.ai/easyocr/>
- [8] ReportLab Inc. (2024). *ReportLab PDF Generation User Guide*. <https://www.reportlab.com/docs/>
- [9] Python Software Foundation. *Regular expression operations (re module)*. Python 3 Documentation.
- [10] Vklinhhh. (2023). *OCR Handwriting Dataset (HAT2023)*. HuggingFace Datasets.